

Universidad de San Carlos de Guatemala  
Facultad de Ingeniería  
Escuela de Ingeniería en Ciencias y Sistemas  
Organización de Lenguajes y Compiladores 2  
Vacaciones del primer semestre 2024  
Catedrático: MSc. Luis Fernando Espino Barrios  
Tutor Académico: José Andres Rodas Arrecis



# Proyecto Fase 3

## ARMv8 Simulator

|                                                     |           |
|-----------------------------------------------------|-----------|
| <b>1. Competencias.....</b>                         | <b>2</b>  |
| 1.1. Competencia General.....                       | 2         |
| 1.2. Competencias Específicas.....                  | 2         |
| <b>2. Descripción.....</b>                          | <b>2</b>  |
| 2.1. Componentes de la aplicación.....              | 3         |
| 2.1.1. Interpretación del código.....               | 3         |
| 2.1.2. Visualización de memoria en tiempo real..... | 3         |
| 2.1.3. Controles de ejecución.....                  | 3         |
| 2.1.4. Reporte de errores.....                      | 3         |
| 2.2. Flujo del proyecto.....                        | 5         |
| 2.3. Arquitectura.....                              | 7         |
| 2.3.1. Aplicación Web Estática.....                 | 7         |
| <b>3. Implementaciones del lenguaje.....</b>        | <b>7</b>  |
| 3.1 Código Assembler ARMv8.....                     | 8         |
| <b>4. Reportes Generales.....</b>                   | <b>11</b> |
| 4.1. Reporte de errores.....                        | 12        |
| 5. Entregables.....                                 | 12        |
| 6. Restricciones.....                               | 12        |
| 7. Consideraciones.....                             | 13        |
| 8. Entrega del proyecto.....                        | 13        |

# 1. Competencias

## 1.1. Competencia General

Implementar nuevas funcionalidades al simulador para el lenguaje de programación **ensamblador ARMv8 de 64 bits** que abarque la fase de análisis e interpretación comprendiendo la estructura del código fuente utilizando herramientas adecuadas para la generación de analizadores léxicos, sintácticos y semánticos.

## 1.2. Competencias Específicas

- Utilizar herramientas léxicas y sintácticas, en específico Peggy en Javascript, para implementar el reconocimiento y análisis del código fuente de ARMv8, permitiendo al estudiante comprender la estructura del lenguaje y su sintaxis.
- Implementar un analizador semántico para la interpretación del lenguaje en sus funcionalidades más básicas.
- Familiarizar al estudiante con las herramientas y estructuras de datos disponibles para la creación de un simulador, proporcionando una comprensión profunda de los procesos involucrados en la traducción y ejecución de programas escritos en lenguaje ensamblador arquitectura ARMv8.

# 2. Descripción

La presente sección proporciona una visión general de la tercera fase del proyecto que se solicita a los estudiantes, centrándose en la continuación del desarrollo del simulador de lenguaje ensamblador arquitectura ARMv8 64 bits.

Esta fase se centra en la continuación de la simulación en donde se tendrá que implementar un analizador semántico para la generación de un intérprete para las instrucciones ya disponibles a partir de la fase 2. Adicionalmente se deberán agregar herramientas de visualización de registros y herramientas de depuración para que el simulador brinde una mejor experiencia al usuario.

Los principales cambios que se deberán aplicar en esta fase son los siguientes:

- Interpretación del código.
- Visualización de memoria en tiempo real.
- Controles de ejecución.
- Reporte de errores semánticos.

## 2.1. Componentes de la aplicación

El objetivo es continuar con el desarrollo del entorno para la creación de aplicaciones en el lenguaje ensamblador ARMv8. Este entorno, también conocido como Simulador, estará diseñado para analizar el código fuente escrito en ARMv8 y proporcionar herramientas esenciales para facilitar el desarrollo de software.

La aplicación debe contar con los mismos componentes de la fase anterior, un apartado para editar código, un área de consola, una sección de reportes, herramientas de carga y descarga de archivos, etc.

Adicionalmente se deben agregar los siguientes componentes:

### 2.1.1. Interpretación del código

En esta fase el simulador tendrá una mayor utilidad al implementar la interpretación del código, teniendo como salida acciones las cuales dependen de cada una de las instrucciones definidas en consola. Las instrucciones que se interpretarán son las definidas en la fase anterior del proyecto, basándonos principalmente en la Quick Reference de la universidad de Washington.

### 2.1.2. Visualización de memoria en tiempo real

La aplicación deberá contar con un área en donde se detalle los movimientos que se realizan a nivel de memoria, como por ejemplo los cambios que se realizan entre los distintos registros, el estado de la memoria y el contenido de la pila de ejecución. Para la implementación del visualizador de memoria podemos guiarnos de la siguiente interfaz de ejemplo: <https://cpulator.01xz.net/?sys=arm>

### 2.1.3. Controles de ejecución

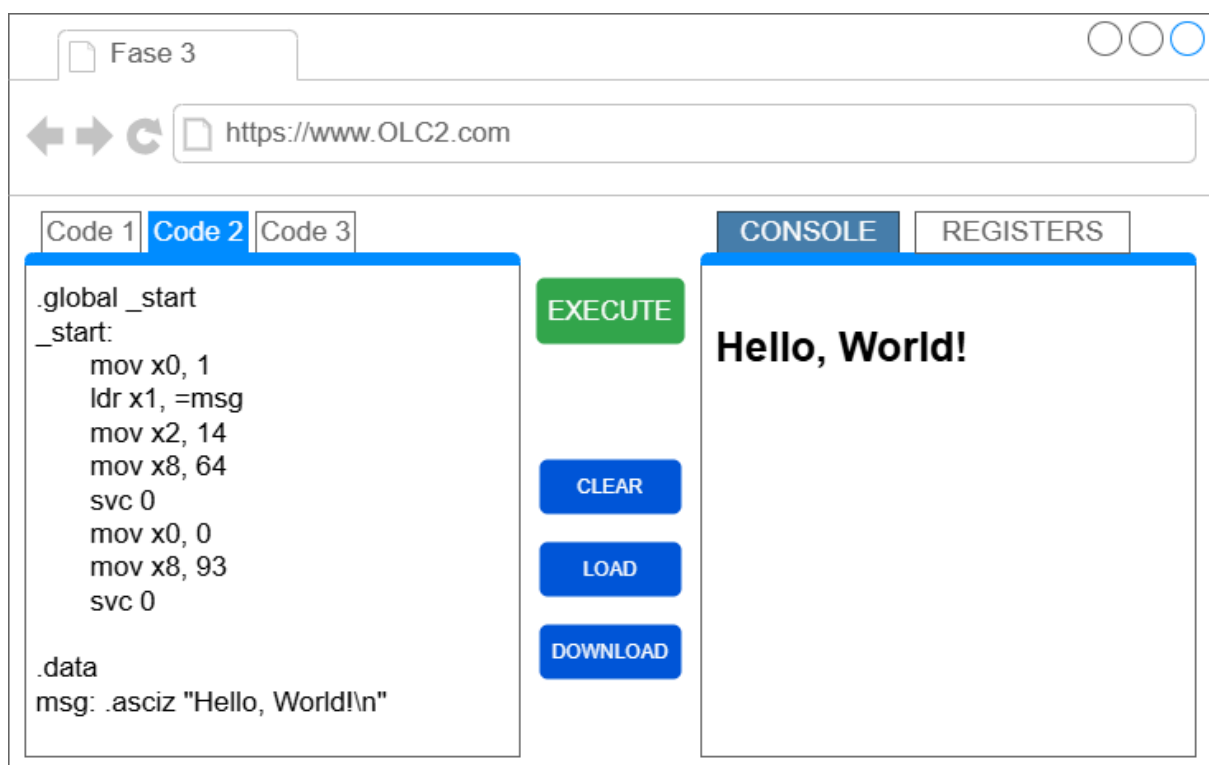
Como una herramienta adicional de desarrollo se debe implementar un área de controles donde se pueda manipular la ejecución del código de entrada. En esta área se debe definir controles que nos permitan realizar la interpretación línea por línea del código así como una interpretación completa. Mientras se vaya ejecutando el código línea por línea se deberán mostrar los respectivos cambios de memoria en la sección anteriormente descrita.

### 2.1.4. Reporte de errores

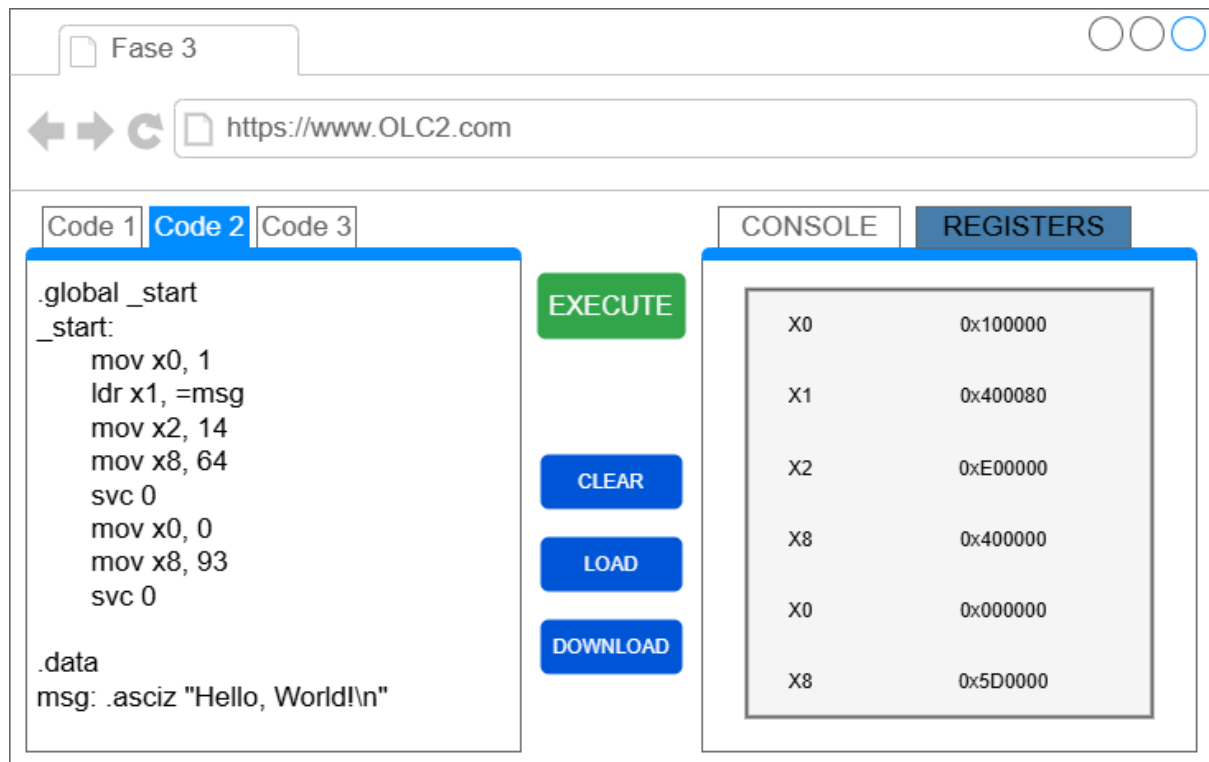
Al finalizar la interpretación del código se debe mostrar en el área de reportes los ya mencionados en las fases anteriores, adicionalmente se debe agregar el manejo de errores semánticos, en donde deberá estar correctamente identificado como en los reportes anteriores.

Debido a que los errores semánticos son completamente manejados por el desarrollador, la aplicación debe contar con recuperación de errores semánticos, por lo que al encontrar un error este se debe almacenar y continuar con las siguientes líneas de análisis.

**Interfaz propuesta:** El simulador deberá contar como mínimo con los siguientes elementos mostrados en la siguiente interfaz.



(Interfaz propuesta)



(Interfaz propuesta)

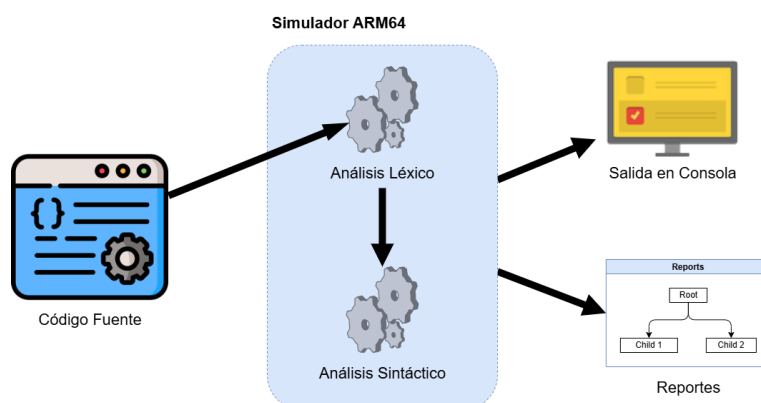
## 2.2. Flujo del proyecto

La ejecución del proyecto sigue un flujo definido según los procesos que los usuarios realizan durante su interacción con el sistema. A continuación, se presenta un desglose paso a paso del flujo del proyecto:

1. Inicio del Programa:
  - El usuario inicia el programa desde la aplicación web.
2. Código Fuente:
  - El usuario tiene la opción de cargar un archivo existente que contiene el código ensamblador o crear uno nuevo desde cero.
  - Si elige cargar un archivo existente, se abre el documento en el editor de texto integrado.
  - Si decide crear un nuevo archivo, se inicia un nuevo documento en blanco en el editor.
  - El código fuente puede contener todas las instrucciones disponibles para el lenguaje ensamblador ARMv8 descritas más adelante.
3. Análisis e interpretación: En el caso del simulador contará con las siguientes etapas del compilador:
  - Análisis Léxico: El simulador escanea el código fuente y lo divide en tokens (unidades léxicas como palabras clave, identificadores, operadores, etc.).

- **Análisis Sintáctico:** Utiliza la gramática del lenguaje ensamblador ARMv8 para verificar la estructura sintáctica del código y construir un árbol de análisis sintáctico (AST).
  - **Análisis Semántico:** Este analizador toma la estructura generada anteriormente y la ejecuta según algún patrón de diseño, dependiendo de cada una de las instrucciones se producirán acciones de salida específicas.
4. **Validación de Resultados:**
    - Dependiendo del código de entrada se mostrarán acciones en la consola de salida.
  5. **Visualización de memoria:**
    - En el área de visualización de memoria se logra apreciar el cambio que han sufrido los registros en sus valores, así como los valores de la pila y la memoria en general.
  6. **Generación de Reportes:**
    - Se generan informes detallados sobre los errores encontrados durante el análisis, incluyendo errores léxicos, sintácticos y semánticos.
    - Adicionalmente se deben mostrar los reportes solicitados en las fases anteriores.
  7. **Fin del Programa:**
    - Una vez completadas todas las etapas, el usuario puede cerrar el programa.

Este flujo del proyecto proporciona una guía clara sobre cómo los usuarios interactúan con la aplicación y cómo se llevarán a cabo los procesos clave durante el desarrollo y análisis del código ensamblador ARMv8.



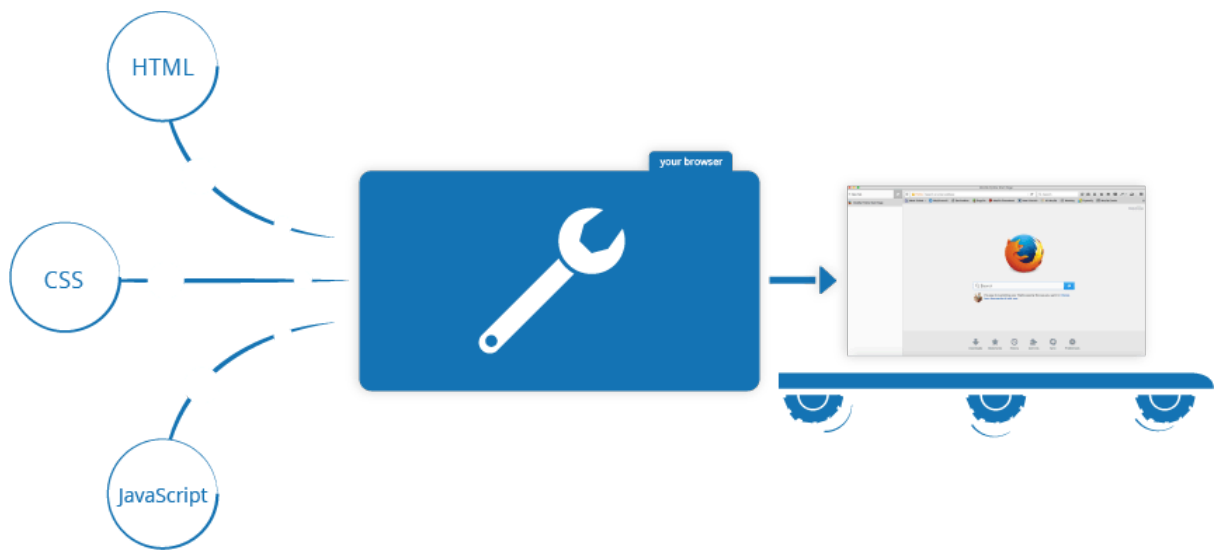
*Flujo de ejecución del proyecto*

## 2.3. Arquitectura

Para la implementación de la solución se requiere la siguiente arquitectura:

### 2.3.1. Aplicación Web Estática

Una aplicación web estática es una aplicación web cuya estructura, contenido y comportamiento no cambian dinámicamente en el lado del servidor. Estas aplicaciones se componen de archivos HTML, CSS y JavaScript que se entregan directamente al navegador del usuario tal como están almacenados en el servidor. Por lo tanto no estará permitido el uso de ningún framework de desarrollo como React, Angular, etc.



## 3. Implementaciones del lenguaje

El lenguaje ensamblador que se evaluará a lo largo del proyecto se conforma por un subconjunto de instrucciones de este. Las instrucciones que se tomarán como válidas para esta fase son las evaluadas en la fase anterior (fase 2).

Para esta fase del proyecto no se deben agregar nuevas producciones en la gramática, ya que únicamente se deberá desarrollar el análisis semántico de la estructura realizada en la fase anterior. Estas instrucciones corresponden a las descritas por la guía rápida (Quick Reference) de lenguaje ensamblador arquitectura ARMv8 de 64 bits descritos por la universidad de Washington.

## 3.1 Código Assembler ARMv8

Las implementaciones del código ensamblador corresponden a las secciones descritas en las primeras dos páginas de la guía rápida de la **Universidad de Washington**, a excepción de las siguientes secciones:

- Cache and TLB Maintenance Instructions.
- DMB and DSB Options.

Para ingresar al documento descrito anteriormente ingresar al siguiente enlace:

- <https://courses.cs.washington.edu/courses/cse469/19wi/arm64.pdf>

Para acceder a la documentación completa de ARMv8 utilizar el siguiente enlace:

- <https://developer.arm.com/documentation/ddi0553/latest/>

Para acceder al listado de llamadas del sistema disponibles utilizar el siguiente enlace:

- [https://chromium.googlesource.com/chromiumos/docs/+/\\_master/constants/syscalls.md#ARMv8-64\\_bit](https://chromium.googlesource.com/chromiumos/docs/+/_master/constants/syscalls.md#ARMv8-64_bit)

### Ejemplo 1:

```
.global _start

_start:
    ldr x0, =str      // load str
    mov x1, 0        // counter = 0
    mov w3, #'a'      // first ascii

loop:
    strb w3, [x0]     // store ascii
    add w3, w3, #1    // next ascii
    add x1, x1, #1    // increment counter
    add x0, x0, #1    // increment offset
    cmp x1, #26       // counter comparison
    bne loop          // goto loop

    mov w3, #10       // new line
    strb w3, [x0]     // store new line

    mov x0, 1         // stdout
    ldr x1, =str      // load str
```



```

    mov x2, 27      // str size
    mov x8, 64      // write syscall_num
    svc 0           // syscall

    mov x0, 0       // return value
    mov x8, 93      // exit syscall_num
    svc 0           // syscall

.data
str: .space 27

```

## Ejemplo 2:

```

.global _start

.data
filename: .asciz "file.txt"
buffer: .asciz "Hello\nWorld!\n"

.text
_start:
    // open file
    mov x0, -100      // open
    ldr x1, =filename // filename address
    mov x2, 101       // O_WRONLY | O_CREATE
    mov x3, 0777      // permissions
    mov x8, 56        // openat
    svc #0            // syscall
    mov x9, x0        // store file descriptor

    // write file
    mov x0, x9        // file descriptor
    ldr x1, =buffer    // buffer address
    mov x2, 14        // size address
    mov x8, 64        // write
    svc 0             // syscall

    // close file
    mov x0, x9        // file descriptor
    mov x8, 57        // close
    svc 0             // syscall

    // exit

```

```
mov x0, 0          // return vale
mov x8, 93         // exit
svc 0              // syscall
```

### Ejemplo 3:

```
.global _start

.data
x: .asciz "Enter a number: "

.bss
y: .space 11
z: .space 34

.text
_start:
    mov x0, 1
    ldr x1, =x
    mov x2, 22
    mov x8, 64
    svc 0
    mov x0, 0
    ldr x1, =y
    mov x2, 11
    mov x8, 63
    svc 0
    mov x0, 0
    ldr x1, =y
    mov x2, 10
    mov x3, 0
loop1:
    ldrb w3, [x1]
    cmp w3, 10
    beq endloop1
    sub w3, w3, 48
    uxtb x3, w3
    mul x0, x0, x2
    add x0, x0, x3
    add x1, x1, 1
    b loop1
endloop1:
    mov x1, 2
    mov x2, 0
```

```

        mov x3, x0
loop2:
        udiv x3, x3, x1
        add x2, x2, 1
        cmp x3, 0
        bne loop2
endloop2:
        ldr x1, =z
        mov x3, 2
        add x1, x1, x2
        add x1, x1, 1
        mov w4, 0
        strb w4, [x1]
        sub x1, x1, 1
        mov x4, 10
        strb w4, [x1]
        sub x1, x1, 1
loop3:
        udiv x4, x0, x3
        msub x5, x4, x3, x0
        add w5, w5, 48
        strb w5, [x1]
        sub x1, x1, 1
        mov x0, x4
        cmp x0, 0
        bne loop3
endloop3:
        mov x0, 1
        add x1, x1, 1
        mov x2, 34
        mov x8, 64
        svc 0
        mov x8, 93
        svc 0

```

## 4. Reportes Generales

Como se indicaba al inicio, el simulador genera una serie de reportes sobre el proceso de análisis de los archivos de entrada. Los reportes son los siguientes:

## 4.1. Reporte de errores

El simulador deberá ser capaz de detectar todos los errores que se encuentren durante el proceso de compilación. Los reportes de errores que se deben mostrar en esta fase son los solicitados en las fases anteriores, adicionalmente se debe agregar el reporte y recuperación de errores **semánticos**.

| No. | Descripción                                          | Línea | Columna | Tipo       |
|-----|------------------------------------------------------|-------|---------|------------|
| 1   | Se esperaba "ID" pero se obtuvo "(".                 | 5     | 1       | sintáctico |
| 2   | Se esperaba "ADD", "SUB" o "MUL" pero se obtuvo "}". | 19    | 6       | sintáctico |
| 3   | El símbolo "¬" no es aceptado en el lenguaje.        | 55    | 2       | léxico     |

## 5. Entregables

El estudiante deberá entregar únicamente el link de un repositorio en GitHub, el cual será privado y contendrá todos los archivos necesarios para la ejecución de la aplicación, así como el código fuente y la gramática. El estudiante es responsable de verificar el contenido de los entregables, los cuales son:

- 5.1. Código fuente de la aplicación
- 5.2. Gramáticas utilizadas

Se deben conceder los permisos necesarios al auxiliar para acceder a dicho repositorio. El usuario es el siguiente:

- AndresRodas

## 6. Restricciones

- 6.1. El proyecto deberá realizarse como una aplicación web estática utilizando el lenguaje **Javascript**.
- 6.2. Es válido el uso de cualquier herramienta y/o librería en Javascript para el desarrollo de la interfaz gráfica a excepción de frameworks que afecten a la arquitectura del proyecto como Angular, Vue.js, React, etc.
- 6.3. Para el analizador léxico y sintáctico se debe implementar una gramática con la herramienta **Peggy** de Javascript.

- 6.4. Para la validación del código ensamblador se debe tener como base la arquitectura ARMv8.
- 6.5. Las copias de proyectos tendrán de manera automática una nota de **0 puntos** y los involucrados serán reportados a la Escuela de Ciencias y Sistemas.
- 6.6. El desarrollo y la entrega del proyecto es en grupos con un máximo de 3 estudiantes.
- 6.7. La distribución de los grupos queda a discreción de los estudiantes.
- 6.8. El sistema operativo queda a elección del estudiante.

## 7. Consideraciones

- 7.1. Durante la calificación se realizarán preguntas sobre el código para verificar la autoría de este, de no responder correctamente la mayoría de las preguntas ***el tutor hará una verificación exhaustiva en busca de copias.***
- 7.2. Se necesita que los estudiantes al momento de la calificación tengan el entorno de desarrollo y las herramientas necesarias para realizar compilaciones, ya que se pedirán pequeñas modificaciones en el código para verificar la autoría de este, en caso que los estudiantes no puedan realizar dichas modificaciones en un tiempo prudencial, los estudiantes tendrán 0 en la sección ponderada y ***el tutor hará una verificación exhaustiva en busca de copias.***
- 7.3. Se tendrá un máximo de 30 minutos por estudiante para calificar el proyecto. La calificación será de manera virtual y se grabará para tener constancia o verificación posterior.
- 7.4. La hoja de calificación describe cada aspecto a calificar, por lo tanto, si la funcionalidad a calificar falla en la sección indicada se tendrá 0 puntos en esa funcionalidad y esa nota no podrá cambiar si dicha funcionalidad funciona en otra sección.
- 7.5. Si una función del programa ya ha sido calificada, esta no puede ser penalizada si en otra sección la función falla o es errónea.
- 7.6. Los archivos de entrada permitidos en la calificación son únicamente los archivos preparados por el tutor.
- 7.7. Los archivos de entrada podrán ser modificados solamente antes de iniciar la calificación eliminando funcionalidades que los estudiantes indiquen que no desarrollaron.
- 7.8. La sintaxis descrita en este documento son con fines descriptivos, el estudiante es libre de diseñar la gramática que crea apropiada para el reconocimiento del lenguaje ensamblador ARMv8.

## 8. Entrega del proyecto

- 8.1. La entrega se realizará de manera virtual, se habilitará un apartado en la plataforma de UEDI para que el estudiante realice su entrega.

- 8.2. No se recibirán proyectos fuera de la fecha y hora estipulada.
- 8.3. La entrega de cada una de las fases del proyecto es en **grupos con máximo de 3 alumnos.**
- 8.4. Fecha límite de entrega del proyecto: **1 de julio de 2024 a las 23:59 hrs.**